

ULTIMATE DOCKER HANDBOOK

YOUR COMPLETE GUIDE TO DOCKER FOR MACHINE LEARNING



BEGINNER
TO ADVANCED

PRACTICAL
EXAMPLES

PRODUCTION
GRADE

INTERVIEW
READY

BUILD. SHIP. SCALE. REPEAT.

Contents

- 01 Docker Fundamentals
- 02 Installation and Core CLI
- 03 Writing a Dockerfile
- 04 Production-Grade Images
- 05 Registries and Tagging
- 06 Volumes and Storage
- 07 Networking and Service Discovery
- 08 Docker Compose
- 09 Docker for Machine Learning
- 10 Local LLMs and MCP
- 11 Security and Production Checklist
- 12 Troubleshooting
- 13 Interview Questions
- MP** Mega Project: House Price Predictor Stack

HOW TO USE THIS HANDBOOK

Read once, revise forever

Each chapter ends with a practice set. Chapters 1 to 8 build the core skills in order; chapters 9 to 13 apply them to real ML systems. The Mega Project combines everything.

Docker Fundamentals

What is a Container?

A container packages an application together with its dependencies and configuration, so it runs the same on a laptop, a CI server, and the cloud. It ends the classic “works on my machine” problem between development, QA and production.

Core Terms

01

Image: read-only layered template (like a class).

02

Container: running instance of an image (like an object).

03

Dockerfile: recipe that builds an image.

04

Registry: image store such as Docker Hub or GHCR.

Docker Architecture

The `docker` CLI (client) sends commands over a REST API to the `dockerd` daemon, which manages images, containers, networks and volumes. If `docker version` shows no **Server** section, the daemon is not running.

Containers vs Virtual Machines

	Container	Virtual Machine
Isolation	Process level, shares host kernel	Full guest OS on a hypervisor
Size and start	MBs, seconds	GBs, minutes
Isolation strength	Weaker (shared kernel)	Stronger

Isolation comes from Linux **namespaces** (what a process can see) and **cgroups** (how much CPU, RAM and I/O it may use). Images are stacked read-only layers; identical layers are cached and shared, which makes builds and pulls fast. On Windows and macOS, Docker Desktop runs containers inside a Linux VM (WSL2 or Hyper-V), so enable hardware virtualization in BIOS.

WHEN TO USE DOCKER FOR ML

Four trigger points

Multiple **machines** (laptop to cloud GPU), multiple **people**, conflicting **library versions**, or moving toward **deployment**. A solo notebook in one clean environment does not need Docker.

Practice Set

1. Explain image vs container using the class/object analogy.
2. Why do containers start faster than VMs?
3. Run `docker version` and identify the client and server sections.

Installation and Core CLI

Install and Verify

Install Docker Desktop (Windows, macOS) or Docker Engine (Linux) from docker.com. On Windows use the WSL2 backend. Verify:

```
docker --version
docker run hello-world      # pulls image if missing, then runs it
```

Essential Commands

Task	Command
Build an image	<code>docker build -t user/app:1.0 .</code>
Run in background	<code>docker run -d --name app -p 8000:8000 user/app:1.0</code>
Throwaway container	<code>docker run --rm -it python:3.11-slim bash</code>
Env var, volume, limits	<code>-e KEY=val -v data:/data --memory 4g --cpus 2</code>
List	<code>docker ps -a</code> <code>docker images</code>
Logs, shell, inspect	<code>docker logs -f app</code> <code>docker exec -it app sh</code> <code>docker inspect app</code>
Stop and remove	<code>docker stop app</code> <code>docker rm -f app</code> <code>docker rmi image</code>
Layers and disk use	<code>docker image history img</code> <code>docker system df</code>
Clean unused	<code>docker system prune</code>

PORT MAPPING

HOST:CONTAINER

`-p 8000:5000` publishes container port 5000 on host port 8000. The app must bind to `0.0.0.0` inside the container; `127.0.0.1` is not reachable from outside it.

Practice Set

1. Run the `docker/getting-started` image on port 80 in detached mode, then stop it.
2. Start a container, open a shell in it with `exec`, and list its filesystem.
3. Find what `docker system prune` would remove before running it.

Writing a Dockerfile

A Dockerfile is a list of instructions the daemon runs top to bottom to build an image. Each instruction adds a layer.

Key Instructions

Instruction	Purpose
FROM	Base image. Always pin the version.
WORKDIR	Working directory for later steps.
COPY	Copy files from build context (prefer over ADD).
RUN	Execute at build time and create a layer.
ENV / ARG	Runtime variable / build-time-only variable.
EXPOSE	Documentation only; -p actually publishes.
USER	Switch to a non-root user.
HEALTHCHECK	Command that reports container health.
ENTRYPOINT / CMD	ENTRYPOINT is the fixed executable; CMD gives default arguments, overridable at <code>docker run</code> . Use exec form <code>["a", "b"]</code> so signals reach your process.

Flask Example

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

```
docker build -t welcomeapp . # '.' is the build context
docker run -p 5000:5000 welcomeapp
```

Why copy requirements.txt first? Docker caches layers. Dependencies change rarely and code changes often, so installing dependencies before copying code avoids reinstalling on every edit.

Practice Set

1. Dockerize a Flask “Hello World” app and run it on host port 8080.
2. Change one line of code and rebuild. Which layers were cached?
3. Difference between CMD and ENTRYPOINT?

Production-Grade Images

01

Slim base: use `python:3.11-slim`. Avoid `alpine` for ML; `musl` breaks many wheels.

02

Pin versions: base image and every package. Never deploy `:latest`.

03

.dockerignore: exclude `.git`, `.venv`, `data`, `.env`.

04

One process per container, logs to `stdout`, config via `env vars`.

Reference ML Serving Dockerfile

```
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ src/
COPY models/ models/ # trained model + preprocessor
RUN useradd -m appuser && chown -R appuser /app
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s CMD python -c \
    "import urllib.request as u;u.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Multi-Stage Build

Compilers and build tools stay in the first stage; only the results reach the final image.

```
FROM python:3.11-slim AS build
RUN apt-get update && apt-get install -y --no-install-recommends build-essential \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip wheel --no-cache-dir -r requirements.txt -w /wheels

FROM python:3.11-slim
COPY --from=build /wheels /wheels
RUN pip install --no-cache-dir --no-index --find-links=/wheels /wheels/* \
    && rm -rf /wheels
WORKDIR /app
COPY src/ src/
USER 1000
CMD ["python", "-m", "src.main"]
```

Merge related shell steps into one `RUN` and clean caches in the same layer; deleting files in a later layer does not shrink the image. Keep serving images free of notebooks, training data and build tools.

Practice Set

1. Add a `.dockerignore` to a project and compare build context size.
2. Convert a single-stage image into multi-stage and compare sizes with `docker images`.
3. Why does a non-root `USER` matter in production?

Registries and Tagging

Image name format: `registry/namespace/repository:tag`.

```
docker login
docker tag app:1.0 docker.io/<user>/app:1.0
docker push docker.io/<user>/app:1.0
docker pull docker.io/<user>/app:1.0
```

- Use **immutable tags** in CI/CD: semantic version or Git SHA (`app:1.4.2`, `app:a1b2c3d`). `latest` is a moving pointer, not a version.
- Tag model images with the model version or MLflow run ID for traceability.
- Use private registries for proprietary models; authenticate CI with access tokens, never your password.
- Building on Apple Silicon for x86 servers: `docker buildx build --platform linux/amd64` .

Practice Set

1. Push an image to Docker Hub with a version tag and pull it on another machine.
2. Why is deploying `:latest` risky?

Volumes and Storage

A container's writable layer is deleted with the container. Persist data outside it.

Type	Syntax	Use for
Named volume	-v pgdata:/var/lib/postgresql/data	Databases, MLflow backend. Docker-managed; survives container removal.
Bind mount	-v ./src:/app/src	Local development, live reload, notebooks.
tmpfs	--tmpfs /tmp	Ephemeral in-memory scratch data.

Format is `SOURCE:TARGET[:mode]`; add `:ro` for read-only (for example, mounting model files into a serving container).

```
docker volume create redis-data
docker volume ls
docker run -d --name cache -v redis-data:/data redis:7-alpine
```

Same in Compose

```
services:
  redis:
    image: redis:7-alpine
    volumes:
      - redis-data:/data      # attach inside the service
volumes:
  redis-data:                # declare at top level
```

Both parts are required: the service-level entry attaches the volume, the top-level entry creates and manages it.

Practice Set

1. Run Redis with a volume, write a key, delete the container, start a new one and confirm the key remains.
2. Mount your local `src/` into a container read-only.

Networking and Service Discovery

- Each container has its own network namespace. `localhost` **inside a container means that container itself**, never a sibling.
- Container IPs are dynamic. Never hardcode them.
- On a user-defined network (Compose creates one automatically), Docker's DNS resolves the **service name** to the container: Streamlit calls `http://fastapi:8000`.
- `ports:` publishes to the host. Container-to-container traffic uses the **container port** and needs no publishing. Keep databases unpublished.
- Plain `docker run` on the default bridge has no name resolution. Create a network: `docker network create ml`, then add `--network ml`.

CLASSIC BUG

UI shows mock data

Streamlit configured with `localhost:8000` cannot reach FastAPI in another container. Fix with an environment variable: `API_URL=http://fastapi:8000`. No code change needed.

Practice Set

1. Create a network, start two containers on it and `ping` one from the other by name.
2. Why does `localhost` fail between containers?

Docker Compose

Compose describes a multi-container app in `compose.yaml`, committed to Git. It is your `docker run` command turned into code. Use `docker compose` (not legacy `docker-compose`); the `version:` key is obsolete. Indent with 2 spaces, never tabs.

Production-Style ML Stack

```
services:
  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.16.0
    command: mlflow server --host 0.0.0.0 --port 5000
    ports: ["5555:5000"]
    volumes: [mlflow-data:/mlruns]
    restart: unless-stopped
  api:
    build: { context: ., dockerfile: Dockerfile }
    ports: ["8000:8000"]
    environment:
      MLFLOW_TRACKING_URI: http://mlflow:5000
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request as u;u.urlopen('http://localhost:8000/health')"]
      interval: 15s
      restart: unless-stopped
  ui:
    build: ./streamlit_app
    ports: ["8501:8501"]
    environment:
      API_URL: http://api:8000
    depends_on:
      api: { condition: service_healthy }
volumes:
  mlflow-data:
```

Command	Effect
<code>up -d [--build]</code>	Create and start; recreates only changed services.
<code>build [svc]</code>	Build images (in parallel).
<code>ps / logs -f [svc]</code>	Status / stream logs.
<code>exec svc sh</code>	Shell in a running service.
<code>stop / start</code>	Stop / start without removing.
<code>down [-v]</code>	Remove containers and network; <code>-v</code> also deletes volumes.

- **Build context** is the directory sent to the builder; all `COPY` paths are relative to it. Use the smallest directory containing everything the Dockerfile needs.
- `depends_on` controls start *order*, not *readiness*. Pair it with healthchecks.
- Per-environment config: `.env`, `env_file:`, or override files with `-f`. Use Compose for dev, test and single hosts; Kubernetes for scale.

Practice Set

1. Write a Compose file for Flask plus Redis and start it with one command.
2. Add a healthcheck and make the UI wait for it.

Docker for Machine Learning

Deployable Unit = Code + Artifacts + Runtime

A `.pkl` file cannot be served directly. Wrap the model **and its preprocessor** in an API (FastAPI) exposing `/predict`, `/batch-predict`, `/health` and `/metrics`, then add a UI (Streamlit or Gradio). The preprocessor must ship with the model so inference applies the same transformations as training.

```
data -> features -> train (log to MLflow) -> model + preprocessor
      -> docker build -> push to registry -> deploy -> monitor -> retrain
```

Orchestrators such as Airflow schedule the recurring steps as DAG tasks.

Where Do Model Artifacts Come From?

Strategy	How	Best for
Bake into image	<code>COPY models/</code>	Small models; immutable and reproducible.
Mount at runtime	<code>-v ./models:/app/models:ro</code>	Large models, frequent retraining.
Download at startup	Pull from MLflow, S3 or HF Hub	Versioned registry workflows.

Never commit generated data or model files to Git; regenerate them with the pipeline or pull them from a registry.

MLflow in a Container

A local `mlflow ui` stores runs in a file on one laptop. A containerized server with a persistent volume or database gives the whole team a shared, durable history; training code points to it with `MLFLOW_TRACKING_URI`.

GPU Workloads

Install the NVIDIA Container Toolkit on the host; the driver stays on the host while CUDA libraries come from the image. Run with `docker run --gpus all`. Match the CUDA base image to your framework build (for example `pytorch/pytorch`).

Deploying to Hugging Face Spaces

1. Create a Space with SDK **Docker**, template Blank. Put the Dockerfile in the repo root; the README header needs `sdk: docker` and `app_port: 7860`.
2. `git remote add hf https://huggingface.co/spaces/<user>/<name>` then `git push hf main`.
3. Spaces run as a **non-root** user: write only to `/tmp` or user-owned paths. A common failure is Gradio flagging writing to a protected directory; fix with `allow_flagging="never"` or `flagging_dir="/tmp/flagged"`. Bind to `0.0.0.0` on the declared port.

Never build images with `docker commit`; it is not reproducible. Use a Dockerfile.

Practice Set

1. Package a scikit-learn model with FastAPI in a Docker image and test `/predict`.
2. Deploy a Gradio app to Hugging Face Spaces using the Docker SDK.

Local LLMs and MCP

Docker Model Runner

Runs LLMs locally as a Docker-managed service (llama.cpp engine, GGUF quantized weights, GPU if available) exposing an **OpenAI-compatible API** (/chat/completions). Models are declared in Compose and pulled from Docker Hub's ai/ namespace.

```
services:
  api:
    build: ./app
    ports: ["8000:8000"]
    models:
      - llm # attach; Docker injects its URL as env var (LLM_URL)
  ui:
    build: ./ui
    ports: ["8501:8501"]
    depends_on: [api]
  models:
    llm:
      model: ai/smollm2 # define the model resource
```

The backend reads the injected URL and posts OpenAI-format JSON: {model, messages:[{role, content}]}. Architecture: **UI** → **FastAPI** → **Model Runner**. Keep the backend layer: it owns auth, rate limiting, prompts, logging and RAG, and lets you swap in a hosted provider by changing one setting. Use Model Runner for local development, offline demos and privacy-sensitive prototyping; use vLLM, TGI or managed APIs for production traffic. The service-level models: attaches; the top-level models: defines (same pattern as volumes).

Docker MCP

MCP (Model Context Protocol) is an open standard that lets AI clients (Claude Desktop, VS Code, Cursor) discover and call tools exposed by MCP servers. Docker runs those servers as isolated containers.

Component	Role
MCP Catalog	Curated directory of ready-to-run servers.
MCP Toolkit	Docker Desktop UI to enable servers and group them into profiles.
MCP Gateway	Runtime that runs servers as containers and gives all clients one endpoint.

Benefits: sandboxing with minimal host privileges, one shared config for every client, managed secrets and OAuth instead of plaintext env vars, and dynamic discovery (mcp-find, mcp-add) mid-conversation.

Practice Set

1. Build a three-layer chat app (UI, FastAPI, Model Runner) in one Compose file.
2. Explain why the UI should not call the model directly.
3. Compare Model Runner with Ollama and vLLM.

Security and Production Checklist

- Run as non-root; avoid `--privileged`; never mount `/var/run/docker.sock` into application containers.
- **No secrets in images, Dockerfiles or Git.** `ENV` and `ARG` values are visible in `docker history`. Inject at run-time (`env_file`, orchestrator secrets, cloud secret managers) or use `BuildKit --mount=type=secret` for build-time needs.
- Use official or verified base images, pinned; rebuild regularly for patches; scan with `docker scout cves` or `Trivy`.
- Set resource limits (`--memory`, `--cpus`) so one model cannot starve the host; set `restart: unless-stopped`.
- Use a read-only root filesystem where possible (`read_only: true` plus `tmpfs` for `/tmp`); drop capabilities (`cap_drop: [ALL]`).
- Expose only required ports; keep databases and trackers on internal networks behind the API or a reverse proxy with TLS.
- Ship `/health`, structured stdout logs and `/metrics` from every service.

RELEASE CHECKLIST

Before you ship

Pinned base · `.dockerignore` · multi-stage · non-root · healthcheck · tagged by version or SHA · scanned · config via `env` · data in volumes.

Practice Set

1. Scan one of your images and list its top vulnerabilities.
2. Find a secret leaked through `docker history` in a sample image and fix the Dockerfile.

Troubleshooting

Symptom	Cause and Fix
Cannot connect to daemon	Daemon not running. Start Docker Desktop or <code>systemctl start docker</code> .
Container exits immediately	Main process ended or crashed. <code>docker logs</code> ; check CMD; debug with <code>sh</code> .
App unreachable from browser	Missing <code>-p</code> or bound to <code>127.0.0.1</code> . Publish port, bind <code>0.0.0.0</code> .
Service A cannot reach B	Used <code>localhost</code> or different networks. Use service name and container port.
<code>FileNotFoundError</code> for model	Artifacts not generated or not copied. Train first; check <code>COPY</code> paths vs build context.
Slow rebuilds	Cache invalidated early. Copy requirements first; add <code>.dockerignore</code> .
Huge image	Full base, leftover caches. Use <code>slim</code> , <code>--no-cache-dir</code> , <code>multi-stage</code> .
Permission denied on HF or K8s	Non-root, read-only path. Write to <code>/tmp</code> ; <code>chown</code> directories.
Data lost after restart	No volume. Mount a named volume; avoid <code>down -v</code> .
Port already in use	Change the host side of <code>-p</code> .
Code change not visible	Image not rebuilt. <code>up --build</code> , or bind mount in dev.

Debug workflow: `docker ps -a`, then `docker logs`, then `docker exec -it ... sh`, then `docker inspect` (IP, mounts, env).

Interview Questions

Image vs container?

An image is an immutable layered template; a container is a running instance with a writable layer on top. Many containers can run from one image.

Container vs VM?

Containers share the host kernel and isolate at process level (small, fast, weaker isolation). VMs run a full guest OS on a hypervisor (heavier, stronger isolation).

How does Docker isolate processes?

Namespaces limit visibility, cgroups limit resources, and a layered union filesystem provides the image.

CMD vs ENTRYPOINT?

ENTRYPOINT defines the executable; CMD supplies default arguments that `docker run` can override. Use `exec` form for correct signal handling.

COPY vs ADD?

COPY copies files. ADD also extracts archives and fetches URLs. Prefer COPY for predictability.

EXPOSE vs -p?

EXPOSE is documentation; `-p` publishes a port to the host.

Why bind to 0.0.0.0?

127.0.0.1 accepts connections only from inside the container itself.

How do containers find each other?

Compose's user-defined network provides DNS by service name. IPs change and localhost is the container itself.

Volume vs bind mount?

Volumes are Docker-managed and portable (production data). Bind mounts map a host path (development).

How do you reduce image size?

Slim base, multi-stage build, `--no-cache-dir`, `.dockerignore`, merge and clean in one RUN, ship only inference dependencies.

How do you handle secrets?

Never bake them into images or Git. Inject at runtime via secrets or env files from a secret manager; use build secrets for build-time needs.

Why not alpine for ML?

musl libc lacks prebuilt wheels for many scientific packages, so builds are slow and brittle. Use Debian slim.

Compose vs Kubernetes?

Compose runs multi-container apps on one host (dev, test, small deployments). Kubernetes orchestrates clusters with scaling, self-healing and rolling updates.

Does depends_on wait for readiness?

Not by default. Use `condition: service_healthy` with a healthcheck.

How do you ship an ML model in a container?

Package code, model and preprocessor behind an API; bake small artifacts or pull versioned ones at startup; tag the image with the model version; add health and metrics endpoints; run non-root with resource limits.

What makes a build reproducible?

Pinned base and packages, Dockerfile-defined steps (not `docker commit`), versioned artifacts and immutable tags.

Mega Project: House Price Predictor Stack

Build the production-like local stack used throughout this handbook: **MLflow** (tracking), **FastAPI** (model serving) and **Streamlit** (UI), all started by Compose.

Architecture

```
User -> Streamlit :8501 -> FastAPI :8000 -> model.pkl + preprocessor
                                MLflow :5555 <- training pipeline logs here
```

Workflow

1. Start MLflow: `cd mlflow && docker compose up -d`; open `localhost:5555`.
2. Create a virtual environment (`uv venv --python 3.11`), install requirements and run the pipeline (data processing, feature engineering, training). Confirm `model.pkl` and `preprocessor.pkl` exist and the run appears in MLflow with MAE and R^2 .
3. Write the Dockerfile for FastAPI (Chapter 4) and one for Streamlit.
4. Write the root `compose.yaml` (Chapter 8): FastAPI build context is the project root so it can reach `models/`; Streamlit context is its own folder.
5. Set `API_URL=http://fastapi:8000` for Streamlit. Run `docker compose up -d --build`.
6. Verify: `/docs` loads, Streamlit returns **real** predictions (not mock data), changing inputs changes the price.
7. Commit `compose.yaml`, Dockerfiles and code. Do **not** commit generated data or model artifacts.

ONBOARDING TEST

Three commands

A new teammate should reach a working system with `git clone`, then the MLflow `up -d`, then the pipeline and `docker compose up -d --build`. If it needs more, the setup is not yet reproducible.

Extensions

- Add healthchecks, a pinned MLflow version and resource limits.
- Move MLflow to a Postgres backend with a named volume.
- Push images to a registry tagged by Git SHA and deploy on a cloud VM.
- Add a Model Runner service (Chapter 10) for an LLM-powered explanation endpoint.